PROJET A3 – TRONC COMMUN

2025-2026

SUPERVISION ET REGULATION DE LA TEMPERATURE D'UNE HABITATION

PARTIE INFORMATIQUE

- DESCRIPTION DETAILLEE -

Version 13.0 mise à jour le mercredi 25 février 2026

BREST :	A. BELMOUHCINE	abdelbadie.belmouhcine@isen-ouest.yncrea.fr
	M. DIVERREZ	morgan.diverrez@isen-ouest.yncrea.fr
	F. LEGRAS	francois.legras@isen-ouest.yncrea.fr
NANTES :	F. BESCHER	florian.bescher@isen-ouest.yncrea.fr
	J.-C. MOREAU	jean-christophe.moreau@isen-ouest.yncrea.fr
CAEN :	W. KADDAH	wissam.kaddah@isen-ouest.yncrea.fr
	S. LE GLOANNEC	simon.le-gloannec@isen-ouest.yncrea.fr

TABLE DES MATIERES

1 Introduction 3

2 Description des blocs 4

2.1 Relevé des températures 4

2.2 Interaction avec l'interface Python 5

2.3 Régulation de la température intérieure 6

2.4 Envoi d'une commande de chauffage 9

2.5 Fin du programme 9

2.6 Simulateur du "mini-environnement" et "échanges thermiques" 9

3 Côté pratique 12

3.1 Environnement de travail 12

3.2 Utilisation de la carte électronique 13

3.3 IHM Python 14

4 Evaluation 16

4.1 Recette 16

4.2 Audit du code 17

4.3 QCM 17

5 Annexe : Structure et fonctions et imposées pour les tests automatiques 18

5.1 Fichier de définition « define.h » 18

5.2 Visualisation de la température 18

5.3 Récupération de la consigne 18

5.4 Régulation 19

5.5 Visualisation de la commande de chauffage 19

5.6 Conseils 20

1 INTRODUCTION

La société "PEDAGIX SOFT", spécialisée dans le développement de logiciels pédagogiques, vous demande de réaliser un pupitre de commande sur micro-ordinateur pour le système conçu par la société "PEDAGIX". Ce pupitre de commande devra mettre en œuvre les fonctionnalités suivantes :

- Relevé des températures intérieure et extérieure via une liaison USB
- Visualisation de la température intérieure et extérieure
- Régulation de la température intérieure basée sur une température de consigne en fonction de différentes lois :
  - Tout ou rien
  - Régulateur PID
- Envoi de commande de chauffage au système (déduit du système de régulation)
- Visualisation du témoin de chauffe

La société "PEDAGIX SOFT" ne possède pas les modules de simulations "mini-environnement" et "échanges thermiques" de la société "PEDAGIX". Elle devra donc mettre en place un système permettant de valider ses propres développements.

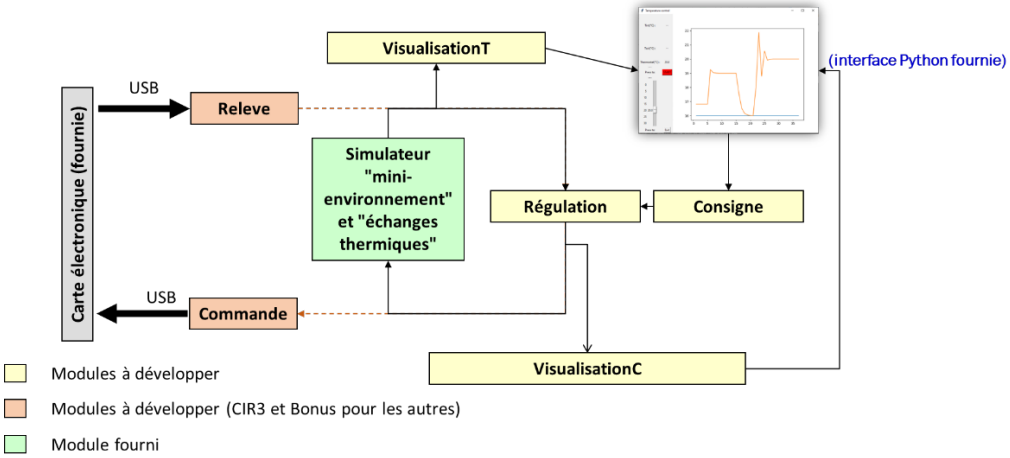


FIGURE 1 : SYNOPTIQUE FONCTIONNEL

2 DESCRIPTION DES BLOCS

2.1 RELEVE DES TEMPERATURES

Le relevé s'effectue via une liaison USB 2.0 entre la carte électronique et le micro-ordinateur de régulation :

- interface de transfert FPGA vers USB par le circuit FT245R de la société FTDI
- protocole de transmission des températures :
  - températures relatives numérisées (SOT) extérieures et intérieures transmises alternativement sur 6 octets.
  - période 40 ms représentant 100 secondes dans la réalité.
  - codage de la température relative numérisée SOT sur 12 bits (codage du capteur conservé) :

Température absolue en °C = -39,64 + 0,04 x SOT

  - format de transfert des températures relatives numérisées SOT : SOT décomposé en quartet et transmis sur 3 octets pour assurer la synchronisation.

	Octet	Bit							
		7	6	5	4	3	2	1	0
Température extérieure	1er	0	0	0	0	SOT Bit 11	SOT Bit 10	SOT Bit 9	SOT Bit 8
	2ème	0	0	0	1	SOT Bit 7	SOT Bit 6	SOT Bit 5	SOT Bit 4
	3ème	0	0	1	0	SOT Bit 3	SOT Bit 2	SOT Bit 1	SOT Bit 0

	Octet	Bit							
		7	6	5	4	3	2	1	0
Température intérieure	1er	1	0	0	0	SOT Bit 11	SOT Bit 10	SOT Bit 9	SOT Bit 8
	2ème	1	0	0	1	SOT Bit 7	SOT Bit 6	SOT Bit 5	SOT Bit 4

	3ème	1	0	1	0	SOT Bit 3	SOT Bit 2	SOT Bit 1	SOT Bit 0

2.2 INTERACTION AVEC L'INTERFACE PYTHON

Une interface utilisateur a été développée en langage Python à votre intention comme illustré en Figure 2.

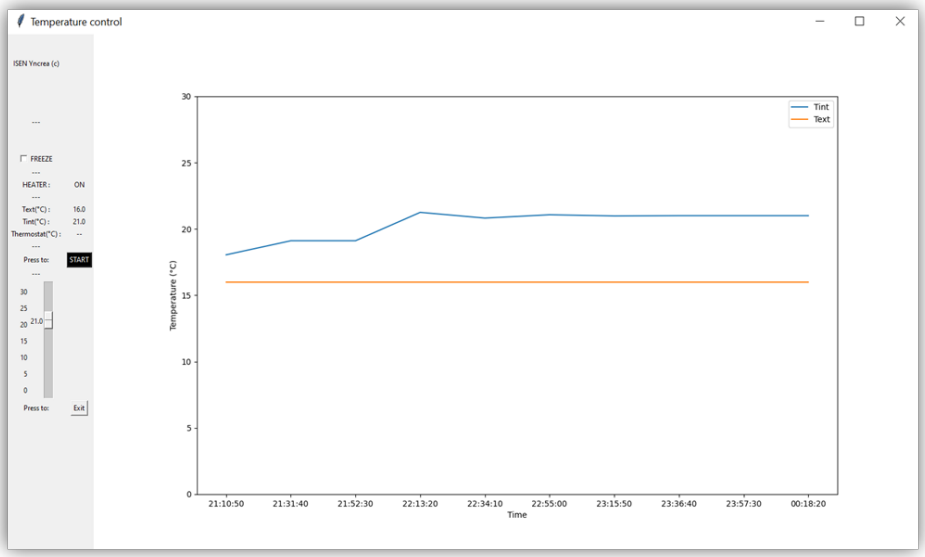


FIGURE 2 : INTERFACE PYTHON

Les données qui apparaissent sur cette interface sont lues dans les fichiers `consigne.txt` et `data.txt` (mise à jour toutes les secondes) comme le montre la Figure 3:

- La consigne doit être comprise entre 5°C et 40°C Cette donnée (type réel) est stockée dans le fichier `consigne.txt`. N'oubliez pas de cliquer sur le bouton Valider pour enregistrer la donnée dans le fichier.
- Les données concernant la température extérieure (type réel), la température intérieure (type réel) et le témoin de chauffe (type chaine de caractères : « true » ou « false ») sont lues dans le fichier `data.txt`. Le témoin de chauffe doit être allumé quand la puissance de chauffage est positive et être éteint quand la puissance de chauffage est nulle. Le fichier `data.txt` comporte trois lignes qui correspondent aux trois données précitées. Dans l'ordre :
    - Temoin (« true » ou « false »
    - Tint (nombre à virgule)
    - Text (nombre à virgule)

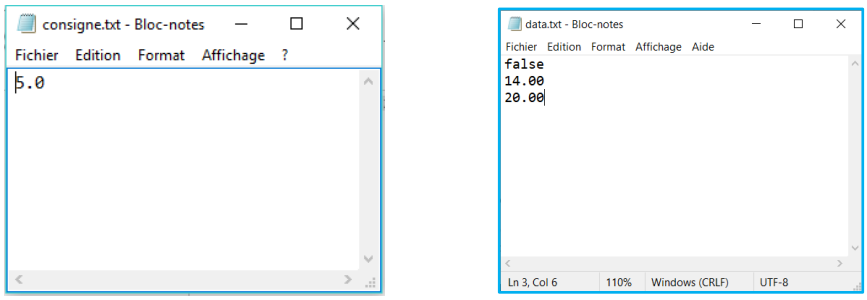


FIGURE 3: EXEMPLE DE FICHIERS CONSIGNE.TXT ET DATA.TXT

Afin d'être sûr de ne pas avoir de problèmes de concurrence entre les programmes, les accès en lecture ou écriture aux fichiers `consigne.txt` et `data.txt` sont protégés par des verrous. Il n'est pas possible de lire et écrire en même temps dans un fichier. C'est ce qu'on appelle l'exclusion mutuelle. Ce mécanisme de verrou est mis en œuvre avec deux fichiers : `.verrouConsigne` et `.verrouData` La présence d'un fichier verrou indique qu'une opération de lecture ou écriture est en cours sur le fichier de données correspondant. L'absence de verrou autorise la lecture ou l'écriture.

2.3 Regulation de la temperature interieure

2.3.1 Tout ou rien

C'est le système le plus simple à mettre en place. On lance une commande de chauffage uniquement si l'on se trouve en dessous de la valeur de la consigne. Le chauffage sera fixé à 50 % lorsqu'il est en marche et à 0 s'il est éteint.

2.3.2 Regulateur PID (Proportionnel Integral Derive)

Un régulateur Proportionnel Intégral Dérivé (PID) est un organe de contrôle permettant d'effectuer une régulation en boucle fermée d'un système industriel. C'est le régulateur le plus utilisé dans l'industrie. Il permet de contrôler un grand nombre de procédés. Nous vous proposons d'utiliser ce type de régulateur pour contrôler le chauffage.

2.3.2.1 Fonctionnement :

Comme son nom l'indique, le régulateur PID est composé de trois termes :

- Proportionnel (P) : évalue la différence entre la mesure et la consigne
- Intégral (I) : évalue la somme des différences précédentes
- Dérivé (D) : évalue la rapidité de changement de la différence

La valeur utilisée pour la régulation PID est la somme de ces trois termes :

$$PID = P + I + D$$

Le poids accordé à ces différents termes permet d'optimiser la régulation du système. On peut dans certains cas mettre l'un des poids à zéro.

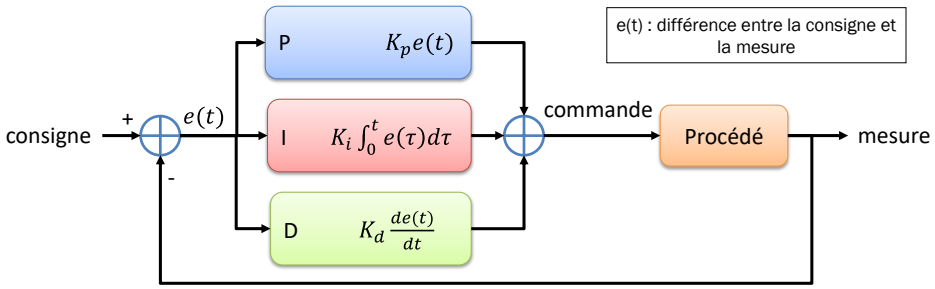
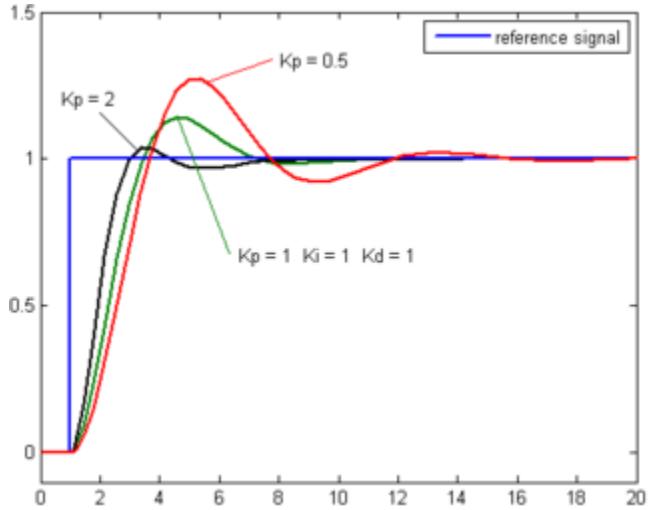


FIGURE 4: ALGORITHME DU REGULATEUR PID

- Terme Proportionnel :

$$P = K_p e(t)$$



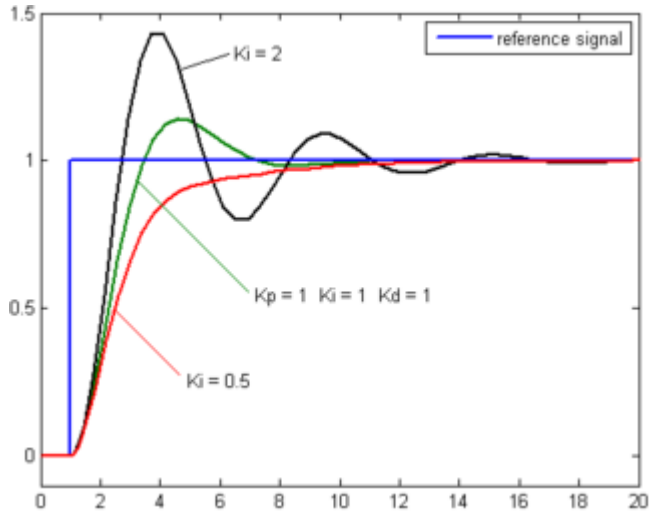
$e(t)$ est la différence entre la consigne et la mesure (dépendant du temps t).

K_p est le gain proportionnel permettant d'ajuster le poids du terme proportionnel.

Un fort gain proportionnel entraîne une réaction forte en sortie. S'il est trop fort le système ne convergera pas. S'il est trop faible, la convergence sera très lente.

- Terme Intégral :

$$I = K_i \int_0^t e(\tau)\, d\tau$$

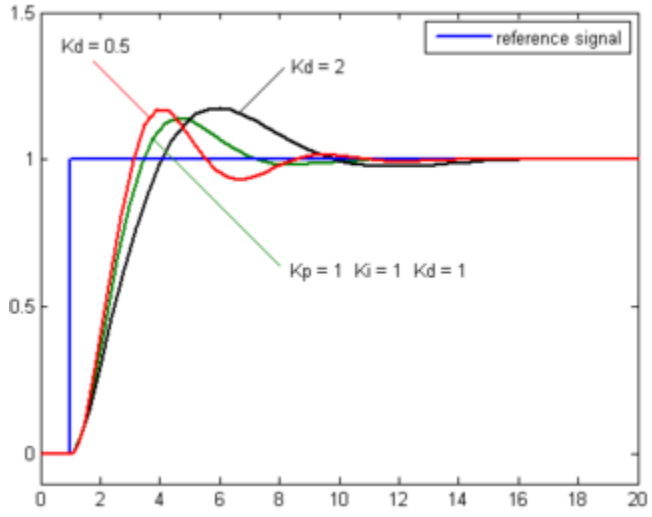


K_i est le gain intégral. Ce terme prend en compte la différence entre la mesure et la consigne ainsi que le temps pendant lequel cette différence existe.

Ce terme accélère la convergence et élimine l'erreur résiduelle introduite par le terme proportionnel.

- Terme Dérivé :

$$D = K_d \frac{de}{dt}$$



K_d est le gain dérivé. Ce terme détermine la pente de la différence par rapport au temps.

Ce terme ralenti les changements du régulateur, ce qui est important lorsque l'on s'approche de la convergence mais ce terme amplifie le bruit.

La complexité de la mise en place d'un système de régulation PID réside dans les points suivants :

- Faire le lien entre le PID et la commande de chauffage. Il faut donc manipuler le PID de façon à avoir une valeur qui variera entre 0 % (pas de chauffage) et 100 % (chauffage maximum)
- Trouver les gains qui optimisent la régulation (manuellement ou en utilisant une méthode telle que celle de Ziegler-Nichols, voir sur Internet)

2.4 Envoi d'une commande de chauffage

L'envoi d'une commande est réalisé via une liaison USB 2.0 avec le micro-ordinateur de régulation :

- interface de transfert USB vers FPGA par le circuit FT245R de la société FTDI
- protocole de commande du chauffage :
  - codage de la puissance de chauffage sur 7 bits :

    puissance en % = (PUIS / 127) x 100

  - codage de chaud /froid sur 1 bit :
    - 0 = chauffage
    - 1 = froid
  - format de transfert de la commande de chauffage :

	Bit							
	7	6	5	4	3	2	1	0
Commande chauffage	0 = chauffage 1 = froid	PUIS Bit 6	PUIS Bit 5	PUIS Bit 4	PUIS Bit 3	PUIS Bit 2	PUIS Bit 1	PUIS Bit 0

2.5 Fin du programme

Lorsque l'utilisateur rentre la valeur de consigne minimum (5°c), **on demande à ce que la régulation s'arrête et que votre programme se termine**. Bien entendu afin de prévenir tout problème pratique (incendie, etc…), vous devez penser à mettre la puissance de chauffage à 0% avant de quitter.

2.6 Simulateur du "mini-environnement" et "echanges thermiques"

La société "PEDAGIX" n'a pas mis sa carte électronique à disposition, il est donc impossible de tester la partie régulation sur le système réel avant la phase d'intégration. Il est ainsi nécessaire de simuler les modules "mini-environnement" et "échanges thermiques" informatiquement.

2.6.1 Principe

Afin de simplifier le développement de votre programme, un simulateur de l'environnement et des échanges thermiques vous est proposé, vous le trouverez sur l'intranet dans la section ressources sous le nom `simulateur.zip`.

Ce simulateur prend en entrée une puissance de chauffage (entre 0 et 100) et ressort une température intérieure et extérieure. En se basant sur une durée de simulation de 10 s de l'environnement, le programme calcule une température intérieure (calculée à partir de la valeur de chauffage) ainsi que la température extérieure (initialisée au début du programme et qui reste constante).

Le simulateur garde une trace du temps écoulé, de la puissance de chauffage demandée, de la température extérieure et de la température intérieure calculée, en écrivant à chaque appel, toutes ces valeurs dans un fichier `trace.txt`. La formule utilisée par le simulateur est la suivante :

$$T_{int}(t+1) = \frac{T_{ext}(t) + R_{GF}\frac{P_{max} \cdot P(t)}{127} + T_{int}(t) \cdot K_r}{K_r + 1}$$

Avec :

$$\begin{cases} P_{max} = 3200 \\ \Delta T = 10 \\ C_{TH} = 73746.9 \\ R_{GF} = 0.00686 \\ K_r = \dfrac{C_{TH} \cdot R_{GF}}{\Delta T} \end{cases}$$

2.6.2 Fonctions

2.6.2.1 Construction du simulateur :

```
struct simParam_s* simConstruct(temp_t temperature)
```

Cette fonction construit une structure de simulateur de type `simParam_s` à partir d'une température extérieure et intérieure initiales stockées dans la variable structurée `temperature`. La fonction initialise également le fichier `trace.txt`. La structure créée est retournée par la fonction.

2.6.2.2 Fonction de simulation :

```
temp_t simCalc(float puissance, struct simParam_s* param_ps);
```

La fonction réalise une simulation de 10 secondes d'échange thermique en prenant en entrée une valeur de puissance représentée par la variable `puissance` et retourne un nouveau couple de températures intérieure et extérieure sous la forme d'une structure de type `temp_t`. La fonction met également à jour le fichier `trace.txt`.

2.6.2.3 FONCTION DE DESTRUCTION DU SIMULATEUR :

```
void simDestruct(struct simParam_s* param_ps);
```

Cette fonction détruit la structure de simulateur.

2.6.3 EXEMPLE D'UTILISATION

- Tout d'abord il faut extraire de l'archive `simuleur.zip`, les fichiers sources `simulateur.c` et `simulateur.h`.
- Vous pouvez ensuite ouvrir le fichier test_simulateur.c qui montre comment utiliser le simulateur.
- Vous pouvez ensuite compiler puis exécuter le code test de la façon suivante :

```
gcc -c simulateur.c

gcc -c test_simulateur.c

gcc simulateur.o test_simulateur.o -o test_simumateur.exe
```

3 COTE PRATIQUE

3.1 ENVIRONNEMENT DE TRAVAIL

Pour la partie simulation, l'environnement de travail n'est pas imposé, veuillez-vous reporter aux consignes données dans le module « Algorithme et Langage C ».

Pour la partie connexion à la carte électronique, les drivers USB sont prévus pour fonctionner sous Windows uniquement. **Un développement sous windows devient ainsi nécessaire**. Nous vous conseillons d'installer MSYS2 ainsi que le compilateur MinGW64 (https://www.msys2.org/). A partir de MSYS2 les installations de gcc et make se font de la façon suivante :

```
pacman -S mingw-w64-ucrt-x86_64-gcc

pacman -S make
```

Afin de faciliter la recette on vous demande d'organiser les fichiers de votre projet de façon à avoir une architecture identique à la Figure 5. Attention toute architecture différente pourra entrainer un malus lors de la recette.

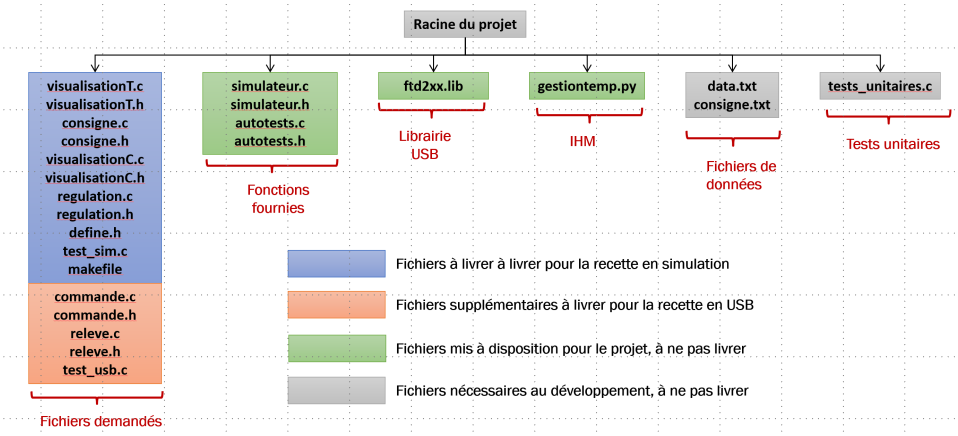


FIGURE 5 : ARCHITECTURE DES FICHIERS (OBLIGATOIRE)

Pour utiliser les drivers USB de la carte FTDI, on vous demande de suivre le guide d'installation donné sur l'intranet. Pour la compilation, il vous sera nécessaire d'inclure les fichiers suivants :

```
#include <windows.h>

#include "ftd2xx.h"
```

De même lors de l'édition de lien il faut ajouter la librairie ftd2xx.lib[1] :

```
gcc toto1.o toto2.o ftd2xx.lib -o toto.exe
```

Attention si l'architecture des fichiers représentée en Figure 5 n'est pas respectée, la commande précédente échouera.

3.2 UTILISATION DE LA CARTE ELECTRONIQUE

3.2.1 INSTALLATION

La carte doit être alimentée en 5V DC et reliée à votre PC au moyen d'un cordon mini USB branché sur le port « USB COM » comme le montre la Figure 6.

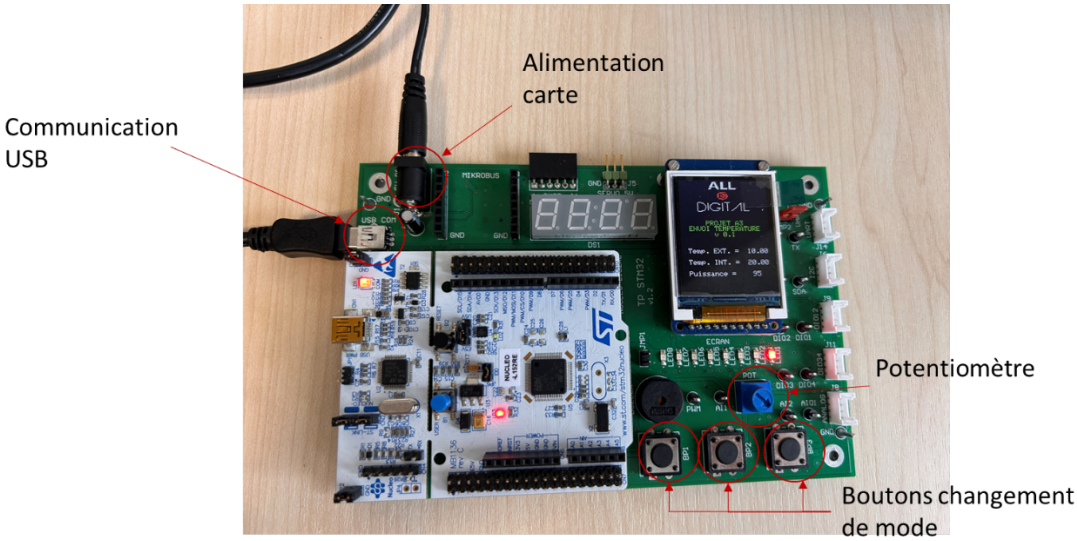


FIGURE 6 : CARTE ELECTRONIQUE DE TEST (PEUT ETRE LEGEREMENT DIFFERENTE)

Il est également possible d'alimenter la carte au moyen d'un second câble mini-USB relié à votre PC en utilisant le port « USB PWR »

---

[1] La librairie à utiliser est \amd64\ftd2xx.lib si MinGW64 est utilisé et \i386\ftd2xx.lib si MINGW32 est utilisé. Attention, en toute logique il faut également recopier dans le répertoire courant le fichier .dll se trouvant dans \amd64 (ou\i386 selon la librairie utilisée) et **le renommer si besoin ftd2xx.dll**. Toutefois windows installe souvent ce fichier .dll lors de la première connexion de la carte.

3.2.2 PARAMETRAGE DRIVER

Lors de l'initialisation de la liaison USB au moyen de la bibliothèque FTD2XX.lib, **il est nécessaire d'effectuer les paramétrages suivants** :

- Débit de la communication (*baudrate*) fixé à **115200** bauds
- Caractéristiques des données
  - **8 bits par mots**
  - **1 bit de stop**
  - **Pas de parité**
- Contrôle de flux
  - **Pas de contrôle de flux**
  - **Autres paramètres fixés à 0**

3.2.3 UTILISATION

Par défaut la carte envoie des températures fixe : $T_{ext} = 10°C$ et $T_{int} = 20°C$

- Appui sur BP1 passe de température extérieure fixe à variable
  - LED 8 sert d'indicateur température extérieure variable avec pot
  - LED 8 OFF : envoie Température valeur fixe $T_{ext} = 10°C$
  - LED 8 ON : envoie Temp ext regle par POTentiometre avec $T_{int} = T_{ext} + 5\ °C$
- Appui sur BP2 passe de simulateur thermique off à simulateur thermique actif
  - LED 6 sert d'indicateur simulateur habitat actif
  - LED 6 OFF : envoie Temperature T_{ext}fixe ou variable et $T_{int} = T_{ext} + 5\ °C$
  - LED 6 ON : envoie Température T_{ext} fixe ou variable et $T_{int} = f(T_{ext}, puiss)$
- Appui sur BP3 passe de fenêtre fermée à fenêtre ouverte
  - LED 4 sert d'indicateur fenêtre fermée/ ouverte
  - LED 4 OFF : fenêtre fermée
  - LED 4 ON : fenêtre ouverte

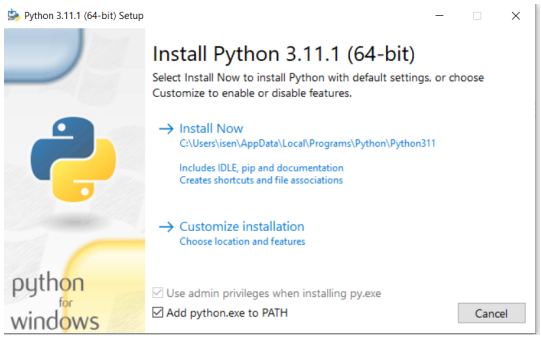
La période d'échantillonnage est 40 ms représentant en réalité 100 s.

3.3 IHM PYTHON

3.3.1 INSTALLATION

- Télécharger python si besoin sur https://www.python.org/downloads/

- Lancer l'installation en pensant à cliquer sur « add python.exe to PATH »



- Ouvrir un terminal (sous windows « invite de commande ») et taper les commandes suivantes

```
python -m pip install -U numpy

python -m pip install -U matplotlib
```

3.3.2 Utilisation

Le principe d'utilisation de l'IHM est donné en Figure 7.

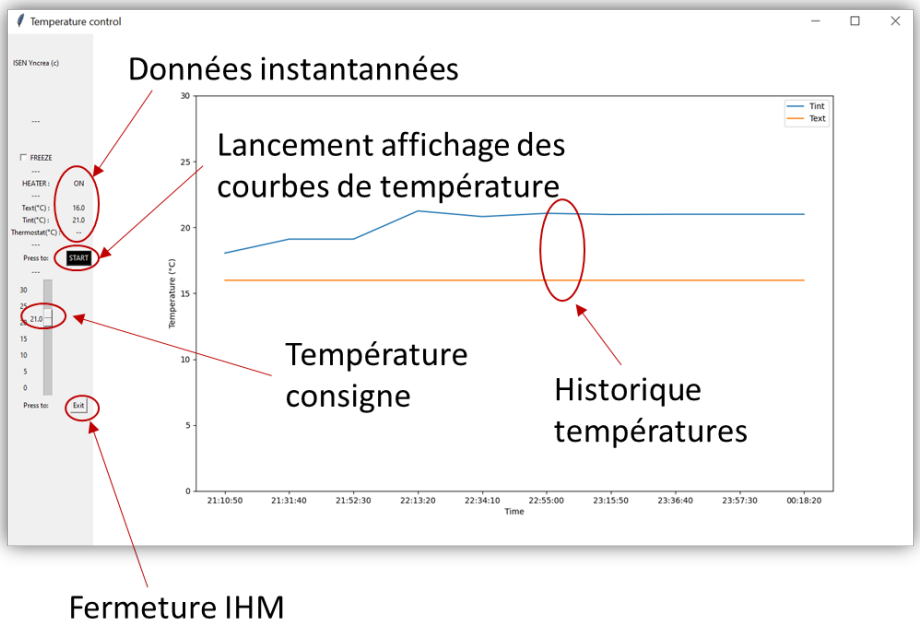


FIGURE 7 : UTILISATION DE L'IHM

4 EVALUATION

Le barème prévisionnel du projet est décrit ci-dessous. Notez que vous serez évalué selon des compétences qui alimenteront votre portfolio. Les coefficients des élèves de CIR3 sont légèrement différents.

			Pondération		
Méthode d'évaluation	Compétences évaluées	Principe de validation	Pondérati	CIR3	test
Recette	L'étudiant·e sait developper des fonctions en respectant un cahier des charges	consigne, VisualisationC, regulationTOR, regulation PID Tests manuels des blocs releve et	4,5	3	0,7
Recette	L'étudiant·e sait intégrer différentes fonctions	Tests manuels en simulation et USB	3	4,5	0
QCM papier	L'étudiant·e maitrise son sujet sur les aspects techniques.	Note de QCM	2	2	0,25
QCM papier	L'étudiant·e maitrise le langage de programmation	Note de QCM	2	2	0,375
QCM papier	L'étudiant·e est capable de prendre du recul sur le sujet du projet	Note de QCM	2	2	0,5
Audit	L'étudiant·e est capable d'expliquer l'entrée/la sortie (le type et le sens) d'une fonction	Questions aux étudiants	1,5	1,5	0
Audit	L'étudiant·e est capable d'expliquer, de manière globale, le fonctionnement global d'une fonction (son déroulé, son utilité globale)	Questions aux étudiants	1,5	1,5	0
Audit	L'étudiant·e est capable d'expliquer une fonction ligne par ligne	Questions aux étudiants	1,5	1,5	0
Audit	L'étudiant·e se soucie de la suite de son projet (documentation...)	Audit de code	1	1	0,5
Audit	L'étudiant·e sait respecter des consignes	Audit de fichiers (Noms des fonctions, nommage des fichiers, makefile, ...)	1	1	1
Total			**20**	**20**	**6,9**

4.1 RECETTE VPL

La recette de votre code se fera de manière automatique par l'outil VPL intégré à moodle (section « Rendu du code »). Au fur et à mesure de vos développements et jusqu'à 11h30 le dernier jour, vous pourrez déposer votre code sur la plateforme afin de le tester. Si votre code compile, il peut être alors évalué par une note sur 100.

Attention (1) de bien respecter les prototypes de fonctions que nous vous fournissons au risque de ne pas pouvoir être évalués et obtenir une note de zéro à la recette.

Attention (2) vous devez *individuellement* déposer votre code pour être évalués. Vous travaillez en binôme, mais vous serez évalués individuellement.

C'est la version finale de votre code qui sera prise en compte pour votre note.

4.2 AUDIT DU CODE

Lors de la recette, l'examinateur auditera votre code sur les points suivants

- Respect du cahier des charges
- Lisibilité du code
- Présence de commentaires
- Absence de variables globales
- Démonstrations (en simulation et en usb)
- Explication de l'ensemble du code par les deux membres du binôme

4.3 QCM

Lors du jour de la recette, une épreuve théorique de validation des connaissances acquises sera effectuée en fin de matinée le dernier jour, sous la forme d'un questionnaire à choix multiple (QCM) d'une durée de 30 min. Ce QCM sera effectué individuellement, sur papier et sans documents.

5 Annexe : Structure et fonctions imposees pour les tests automatiques

5.1 Fichier de definition « define.h »

```
#ifndef DEFINE_H
#define DEFINE_H

typedef struct{
          float exterieure;
          float interieure;
} temp_t;

…

#endif
```

5.2 Visualisation de la temperature

visualisationT.h

```
#include "define.h"
void visualisationT(temp_t maTemperature);
```

visualisationT.c

```
void visualisationT(temp_t maTemperature){
          …
          …
}
```

Attention, si le verrou est présent, la fonction ne doit pas modifier le fichier `data.txt`.

5.3 Recuperation de la consigne

consigne.h

```
float consigne(float csgn);
```

consigne.c

```
float consigne(float csgn){
  …
  return csgn;
}
```

Attention, si le verrou est présent la fonction doit retourner le paramètre `csgn` passé en entrée de la fonction !

5.4 REGULATION

Ce bloc est un peu particulier car le prototype de la fonction régulation n'est pas imposé, ceci dans le but de laisser au libre choix des étudiants les paramètres d'entrées et de sorties du bloc. En revanche, il est demandé de fournir une fonction de test de ce bloc régulation nommée « `regulationTest` » dont le prototype est imposé :

regulation.h

```
float regulationTest(int regul,float csgn,float* tabT, int nT);
```

regulation.c

```
float regulationTest(int regul,float csgn,float* tabT, int nT){
        float cmd=100.0;
        …
        …
        return cmd;
 }
```

Cette fonction retourne la valeur de la commande de chauffage obtenue après nT appels à la fonction `regulation()` :

- `regul` : type de régulation (1:Tout ou Rien, 2:PID)
- `csgn` : température de consigne
- `tabT` : tableau de températures intérieures successives en entrée de la régulation
- `nT` : nombre d'entrées dans le tableau `tabT`

Pour la régulation PID, les gains sont fixés à Kp=1.1, Ki=0.2 et Kd=0.15. La loi de saturation entre la valeur de PID et la commande de chauffage est linéaire.

On considèrera que, comme la dérivé et l'intégrale n'ont pas de sens à la première itération, alors les termes D et I du PID ne sont pas pris en compte à la toute première itération.

5.5 VISUALISATION DE LA COMMANDE DE CHAUFFAGE

visualisationC.h

```
void visualisationC(float puissance);
```

visualisationC.c

```
void visualisationC(float puissance){
        …
        …
}
```

Si le verrou est présent, la fonction ne modifie pas le fichier `data.txt`. Le paramètre puissance représente la puissance de chauffage en % allant de 0 à 100

5.6 CONSEILS

Une archive contenant tous les fichiers demandés et les prototypes associés vous est fournie sur moodle sous le nom minimal.zip.

Nous vous conseillons de partir de ces fichiers et de compléter au fur et à mesure les différentes fonctions.

Si vous n'avez pas réussi un bloc, commentez le code qui n'est pas fonctionnel mais laissez le squelette obligatoire de la fonction (coquille vide).